ROTEIRO CI EXPERT
PROJETO E IMPLEMENTAÇÃO DE PROCESSADOR MULTICICLO

1. Divisão do Grupo e Atribuições por Trilha

Este roteiro organiza as atividades práticas do trabalho final em grupo no âmbito do CI Expert. O objetivo principal do trabalho é projetar, verificar, sintetizar e realizar a implementação física de um processador multiciclo parametrizado, garantindo o fechamento completo do fluxo de circuitos integrados digitais (RTL to Sign-Off).

Para assegurar uma abordagem profissional e representativa da indústria de semicondutores, a equipe será composta por integrantes das 4 trilhas da parte digital (RTL Synthesis, Design for Test, Design Verification e Physical Design), onde cada aluno assume a liderança e responsabilidade técnica de uma área específica do fluxo EDA, conforme ilustrado no diagrama geral a seguir:

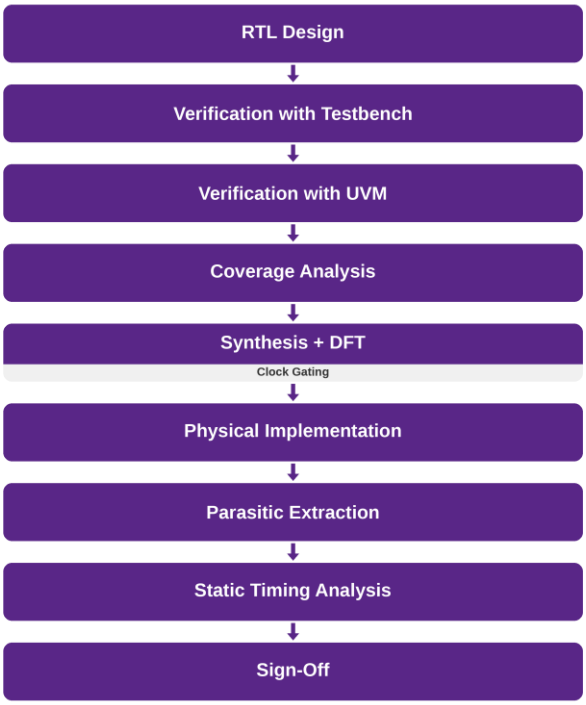


Figura 1: Fluxo Integrado de Desenvolvimento de Circuitos Integrados Digitais (RTL to Sign-Off).

2. Descrição Detalhada da Microarquitetura Multiciclo

A microarquitetura a ser desenvolvida consiste em um processador multiciclo de 3 estágios sem pipeline. A execução de qualquer instrução é distribuída ao longo de 3 ciclos de relógio consecutivos (Fetch/Decode, Execute, Store), além do estado inicial de Reset.

2.1 Visão Geral e Diagrama Estrutural do Datapath

O datapath é parametrizável na largura do barramento de dados (parâmetro WIDTH, valor padrão = 8 bits). A figura a seguir apresenta a interconexão estrutural completa entre os módulos combinacionais, registradores e a unidade de controle:

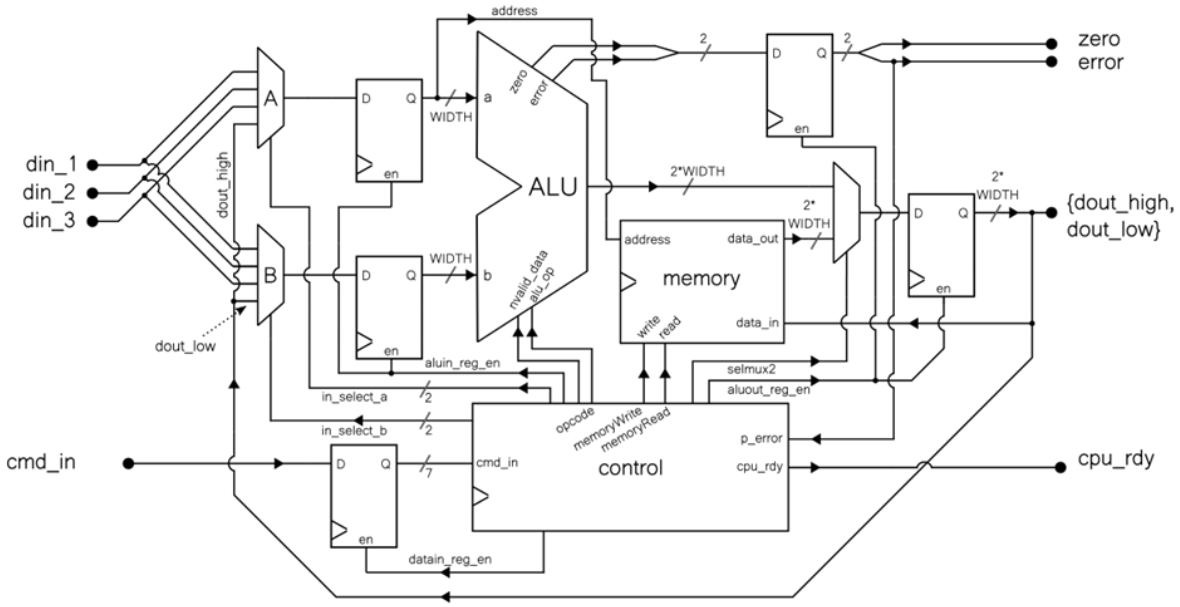


Figura 2: Diagrama de Blocos Estrutural da Microarquitetura e Datapath do Processador Multiciclo.

Componentes e Blocos da Microarquitetura:

- **1. Multiplexadores A e B (mux4 / mux4_registered):** Selecionam os operandos das entradas da ALU a partir dos barramentos externos de entrada (din_1, din_2, din_3) ou da realimentação do resultado anterior (dout_high no MuxA e dout_low no MuxB). As saídas dos multiplexadores são capturadas em registradores temporários (regA e regB) ativados pelo sinal de controle aluin_reg_en.
- **2. Banco de Registradores (register_bank):** Módulos síncronos na borda de subida do relógio (clk), com reset assíncrono (rst) e habilitação de escrita (wr_en/reg_en). Armazenam os operandos da ALU, a instrução de comando (reg_cmd), as flags de status (reg_flags) e a palavra de saída final.
- **3. Unidade Lógica e Aritmética (ALU):** Executa operações aritméticas e lógicas. A ALU aceita dois operandos de largura WIDTH e produz um resultado de 2 x WIDTH bits. Emite a flag zero (quando o resultado é 0) e a flag error (em caso de divisão por zero ou quando dados de realimentação forem inválidos). Sob condição de erro, a saída da ALU é obrigatoriamente forçada para -1 (todos os bits em '1').
- **4. Memória Interna (memory):** Memória interna estática com capacidade de **8 palavras** de largura 2 x WIDTH bits. Apresenta escrita síncrona habilitada por memoryWrite e leitura assíncrona habilitada por memoryRead.

- **5. Registrador de Saída (reg_out):** Armazena a palavra computada de 2 x WIDTH bits (ativado por aluout_reg_en). Os bits mais significativos formam o barramento externo dout_high e os menos significativos formam dout_low.
- **6. Unidade de Controle (control):** Decodifica a palavra cmd_in de 7 bits e emite os sinais de habilitação de registradores, seleções de MUX, sinais de leitura/escrita de memória e o indicador de prontidão cpu_rdy.

2.2 Conjunto de Instruções (ISA) e Formato do Barramento cmd_in[6:0]

A operação do processador é governada por uma palavra de controle externa de 7 bits (cmd_in[6:0]), cuja divisão de campos e codificação estão ilustradas a seguir:

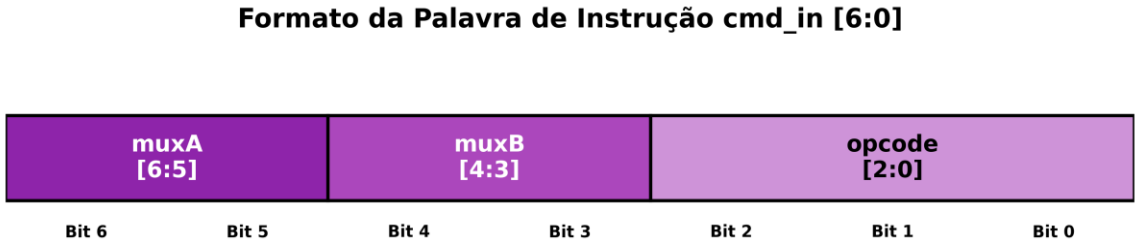


Figura 3: Layout de Bits do Barramento de Comando cmd_in[6:0].

Campo	Bits em cmd_in[6:0]	Descrição da Função
muxA	cmd_in[6:5]	Seleção da fonte do Operando A (00: din_1, 01: din_2, 10: din_3, 11: dout_high)
muxB	cmd_in[4:3]	Seleção da fonte do Operando B (00: din_1, 01: din_2, 10: din_3, 11: dout_low)
opcode	cmd_in[2:0]	Decodificação da operação da ISA executada pela ALU ou Memória

Tabela de Decodificação do Opcode (ISA):

opcode[2]	opcode[1]	opcode[0]	Instrução	Ação no Datapath
0	0	0	ADD	Soma: Out = A + B
0	0	1	SUB	Subtração: Out = A - B
0	1	0	MUL	Multiplicação: Out = A x B
0	1	1	DIV	Divisão: Out = A / B (Erro se B = 0)
1	0	0	NOP	Nenhuma Operação
1	0	1	LOAD	Leitura da Memória Interna
1	1	0	STORE	Escrita na Memória Interna
1	1	1	NOP	Nenhuma Operação

2.3 Unidade de Controle e FSM

A FSM de controle opera em 4 estados principais dispostos conforme a Figura 4:

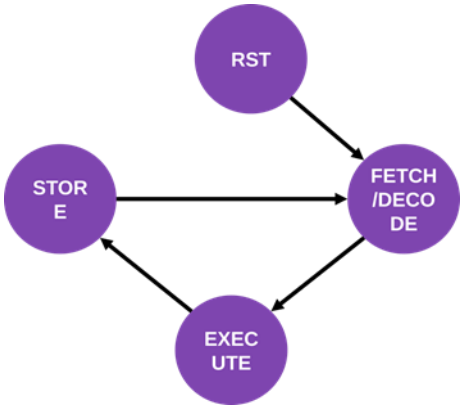


Figura 4: Máquina de Estados Finitos (FSM) da Unidade de Controle.

Detalhamento dos Ciclos da FSM:

- **1. RST (Estado 2'b00):** Estado de inicialização ativado durante o reset assíncrono (rst). Todos os registradores e saídas são forçados a zero; cpu_rdy permanece desativado (0).
- **2. FETCH / DECODE (Estado 2'b01):** 1º Ciclo da Instrução. A Unidade de Controle ativa cpu_rdy = 1 para indicar que está pronta para receber o novo comando cmd_in. Ativa aluin_reg_en = 1 (carregando operandos nos registradores regA e regB) e datain_reg_en = 1 (armazenando o comando em reg_cmd). Se os operandos utilizarem realimentação (dout_high/dout_low) e o resultado anterior tiver flag error, ativa nvalid_data = 1.
- **3. EXECUTE (Estado 2'b10):** 2º Ciclo da Instrução. Desativa cpu_rdy = 0. A ALU processa os dados de acordo com o opcode ou a memória executa a leitura assíncrona (memoryRead = 1) caso seja instrução LOAD.
- **4. STORE (Estado 2'b11):** 3º Ciclo da Instrução. Ativa aluout_reg_en = 1 para armazenar o resultado final no registrador reg_out e gravar na memória (memoryWrite = 1) se for instrução STORE. O controle adquire a próxima palavra cmd_in e retorna automaticamente ao estado FETCH / DECODE.

> **Resumo dos Sinais Gerados pela FSM**
> • aluin_reg_en: Habilita os registradores de entrada da ALU (regA e regB)
> • datain_reg_en: Habilita o registrador do comando cmd_in (reg_cmd)
> • aluout_reg_en: Habilita o registrador de saída (reg_out) e de flags (reg_flags)
> • cpu_rdy: Indica para o ambiente externo que o processador está pronto para nova instrução
> • memoryWrite / memoryRead: Controles de escrita e leitura síncrona/assíncrona na memória
> • selmux2: Seleciona se a saída para reg_out vem da ALU (0) ou da Memória (1)
> • nvalid_data: Alerta a ALU sobre tentativa de reuso de dados sob condição de erro prévio
> • in_select_a / in_select_b / opcode: Barramentos de seleção dos Muxes e operação da ALU

3. Metodologia de Execução

A execução do projeto segue um fluxo rigoroso por ferramentas Synopsys, garantindo a rastreabilidade da descrição RTL até o Sign-Off Físico.

3.1 Parte 1: RTL Design & Arquitetura

- **Escopo Principal:** Desenvolver a descrição Verilog sintetizável e parametrizada da microarquitetura do processador multiciclo, garantindo conformidade estrita com o conjunto de instruções (ISA) e a máquina de estados finitos (FSM).
- **Atribuições Detalhadas:** Codificar os blocos estruturais parametrizados (Mux 4:1, Mux 4:1 registrado, Banco de Registradores com reset assíncrono e enable, Memória síncrona de escrita e assíncrona de leitura, ALU aritmética e lógica).
- **Atribuições de Código:** Implementar a Unidade de Controle (control.v) utilizando FSM de 2 blocos sempre (síncrono para estado e combinacional para saídas). Estruturar o módulo Top-Level (top.v) conectando o Datapath e o Controle.
- **Ferramentas EDA:** Synopsys VCS.
- **Entregáveis Obrigatórios:** Arquivos Verilog sintetizáveis (top.v, control.v, alu.v, memory.v, register_bank.v, mux4.v, mux4_registered.v) validados e sem erros de compilação.

3.2 Parte 2: Design Verification

- **Escopo Principal:** Garantir a funcionalidade completa do processador e validar a cobertura do projeto utilizando simulação orientada a eventos e a metodologia UVM (Universal Verification Methodology).
- **Atribuições Detalhadas:** Desenvolver o Testbench funcional regular para análise e depuração visual de formas de onda no Synopsys Verdi.
- **Arquitetura UVM:** Construir o ambiente UVM completo contendo Interface, Virtual Interface, Sequencer, Driver, Monitor, Predictor (modelo de referência em nível de transação) e Scoreboard.
- **Cenários de Teste:** Desenvolver e executar suítes de teste cobrindo operações individuais e o teste de estresse ostensivo com 100 mil instruções aleatórias (100k Random Operations).
- **Cobertura e Asserções:** Implementar o coletor de cobertura funcional e cruzada (Cross Coverage entre cmd_in x error e cmd_in x zero), além de codificar asserções SystemVerilog (SVA) para verificar propriedades da FSM (ex.: validação de que cpu_rdy desce a 0 após o início da instrução).
- **Ferramentas EDA:** Synopsys VCS, Synopsys Verdi (Waveform & Coverage Viewer).
- **Entregáveis Obrigatórios:** Ambiente UVM funcional, scripts de simulação, relatórios de cobertura funcional/cruzada e log de simulação zerado sem erros (UVM_ERROR: 0).

3.3 Parte 3: Logic Synthesis & Design for Testability - DFT

- **Escopo Principal:** Mapear a descrição RTL em uma netlist de portas lógicas, inserir estruturas de teste (Scan Chains), otimizar o PPA (Power, Performance, Area) e comprovar a equivalência formal.
- **Restrições SDC:** Configurar o arquivo de restrições de tempo SDC (.sdc) definindo período de relógio, incerteza do clock, transições, latências e atrasos de entrada/saída.

Clock period	30 ns
Clock setup uncertainty	10%
Clock transition	10%
Clock source latency	5%
Clock network latency	3%
Input delay	40%
Output delay	50%
Output load	0.04 pF
Input min transition	1%
Input max transition	10%

- **Fluxo de Compilação:** Executar o fluxo de síntese no Design Compiler inserindo Clock Gating e cadeias de varredura (DFT Scan Chains).
- **Análise de PPA:** Mapear o caminho crítico do circuito (grupo reg->reg) e realizar a exploração de PPA (PPA Sweep), comparando 3 estratégias de compilação: Baseline (clock period igual 30ns), PPA1 (clock period igual 20ns) e PPA2 (clock period igual 10ns).
- **Ferramentas EDA:** Synopsys Design Compiler/Fusion Compiler.
- **Entregáveis Obrigatórios:** Netlist sintetizado com DFT, arquivo de restrições SDC, relatórios de QoR/Power e dados numéricos de PPA.

3.4 Parte 4: Physical Design & Sign-Off

- **Escopo Principal:** Realizar o layout físico do circuito integrado (Placement & Routing) no Fusion Compiler utilizando o PDK de tecnologia do curso, fechar verificações físicas e conduzir o STA Sign-off.
- **Floorplan e Power:** Executar o Floorplanning e estruturação da rede de alimentação (Power Network Rings/Straps VDD/VSS).
- **Fluxo de P&R:** Conduzir as etapas de Posicionamento (place_opt), Síntese da Árvore de Clock - CTS (clock_opt) e Roteamento Automático (route_auto, optimize_routes, route_detail).
- **Verificação Física:** Garantir fechamento físico executando check_lvs -open_reporting detailed e check_design -checks routes até alcançar zero curtos (shorts) e zero conexões abertas (opens).
- **Sign-off RC & STA:** Extrair parasiticos RC via Synopsys STARRC (gerando arquivo SPEF) e realizar a Análise Estática de Tempo Sign-off no Synopsys PrimeTime, construindo o gráfico de convergência de frequência da síntese ao post-route.
- **Ferramentas EDA:** Synopsys Fusion Compiler, Synopsys STARRC, Synopsys PrimeTime.
- **Entregáveis Obrigatórios:** Layout final (DEF/GDSII), relatórios de DRC/LVS zerados, relatórios de timing sign-off e curvas de convergência de frequência.

4. Estrutura Obrigatória da Apresentação Final

A apresentação final do projeto terá a duração exata de 20 minutos. A tabela a seguir especifica o conteúdo obrigatório e a atribuição para cada integrante do grupo:

Seção	Conteúdo Obrigatório Exigido	Responsável
Delivery Requirements	Declaração explícita dos objetivos e lista dos entregáveis submetidos (RTL, Testbenches, Scripts Tcl, Relatórios QoR/Power, Layout e PPA).	Todo o Grupo
Team Introduction	Apresentação dos integrantes, papéis desempenhados (RTL, DV, DFT, PD) e motivações do aprendizado.	Todo o Grupo
Project Implementation & RTL	Apresentação da arquitetura proposta, esquemáticos extraídos do Verdi, código Verilog da FSM e dos blocos do datapath.	Integrante RTL
Verification (UVM)	Estrutura do ambiente UVM (Sequencer, Driver, Monitor, Predictor, Scoreboard), resultados dos testes (100k random), telas de Cobertura Funcional e Asserções SVA no Verdi.	Integrante DV
Logic Synthesis & DFT	Relatórios de QoR e Power do Design Compiler, análise do caminho crítico (reg->reg), detalhamento de Clock Gating e Scan Chains (DFT).	Integrante DFT
Physical Synthesis (P&R)	Imagens do Layout final, relatórios de verificação física LVS/DRC zerados e análise de tempo sign-off.	Integrante PD
PPA Sweep & Trade-Offs	Análise comparativa dos perfis Baseline (potência), PPA1 (performance) e PPA2 (balanceado). Discussão sobre frequência máxima alcançada vs. consumo elétrico.	Todo o Grupo

5. Checklist de Submissão Final para a Equipe

Antes de efetuar a entrega final dos arquivos e realizar a apresentação, o grupo deve validar os seguintes itens de controle de qualidade:

- Códigos Verilog estruturados e sem erros de compilação sintática.
- Log do ambiente UVM finalizado com sucesso e sem nenhum erro (UVM_ERROR: 0) no teste de 100k instruções.
- Métricas de Cobertura Funcional e Cruzada (Cross Coverage) documentadas em imagens extraídas do Verdi.
- Checagem de DRC e LVS zerada (0 shorts e 0 opens).
- Análise de PPA Sweep contendo dados numéricos comparativos de Frequência (MHz) e Consumo de Potência (mW).
- Apresentação em slides sincronizada rigorosamente com o tempo de 20 minutos.